

1. Fundamental Concepts of DBMS

- **Purpose of Database Systems:** Manage large amounts of structured data efficiently, ensure data integrity, provide concurrent access, and enable data recovery.
- **Data Abstraction:**
 - **Physical Level:** Describes how data is actually stored (e.g., files, indices).
 - **Conceptual (Logical) Level:** Describes what data is stored and the relationships among data (e.g., tables, attributes).
 - **External (View) Level:** Describes part of the database relevant to a particular user or application.
- **Data Models:** Collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. Examples: Relational, E-R, Object-oriented.
- **Database Languages:**
 - **DDL (Data Definition Language):** Define schema (e.g., CREATE TABLE, ALTER TABLE).
 - **DML (Data Manipulation Language):** Access and manipulate data (e.g., SELECT, INSERT, UPDATE, DELETE).
 - **DCL (Data Control Language):** Manage permissions (e.g., GRANT, REVOKE).
 - **TCL (Transaction Control Language):** Manage transactions (e.g., COMMIT, ROLLBACK).
- **Database Users:** Application Programmers, Sophisticated Users, Naive Users, Database Administrators.
- **Database Manager:** Software module that interfaces with file system, responsible for data storage, retrieval, and update.
- **Database Administrator (DBA):** Person responsible for overall control of the database system.
- **DBMS Structure:** Query Processor, Storage Manager, Transaction Manager, Concurrency Control, Recovery Manager.

2. Entity-Relationship (E-R) Model

- **Entity Sets:** Collection of similar entities (e.g., STUDENT, COURSE). Represented by rectangles.
- **Relationship Sets:** Association among several entity sets (e.g., ENROLLS between STUDENT and COURSE). Represented by diamonds.
- **Attributes:** Properties of entities or relationships (e.g., student_id, course_name). Represented by ovals.
- **Mapping Constraints (Cardinality Ratios):**
 - One-to-One (1:1)
 - One-to-Many (1:N)
 - Many-to-One (N:1)
 - Many-to-Many (M:N)
- **Keys:**
 - **Super Key:** Set of attributes that uniquely identify a tuple.
 - **Candidate Key:** Minimal super key.
 - **Primary Key:** Candidate key chosen by the designer.
 - **Foreign Key:** Attribute(s) in one table that refer to the primary key of another table.
- **E-R Diagrams:** Graphical representation of the E-R model.
- **Strong and Weak Entity Sets:**
 - **Strong Entity:** Has a primary key of its own.
 - **Weak Entity:** Does not have sufficient attributes to form a primary key; depends on a strong entity (identifying owner) for its existence and identification. Represented by double rectangles, identifying relationship by double diamonds.
- **Extended E-R Features:**
 - **Specialization:** Top-down process of forming sub-entity sets (e.g., PERSON → EMPLOYEE, CUSTOMER).
 - **Generalization:** Bottom-up process of forming a higher-level entity set from lower-level entity sets.
 - **Aggregation:** Treating a relationship set as an entity set for participation in other relationships.

3. Relational Model

- **Structure of Relational Databases:** Data organized into tables (relations). Each table has rows (tuples) and columns (attributes).
- **Database Schema:** Logical design of the database (table names, attribute names, data types, constraints).
- **Query Languages:**
 - **Relational Algebra:** Procedural query language.
 - **Fundamental Operations:**
 - Selection (σ): Filters rows. Example: $\sigma_{salary > 50000}(EMPLOYEE)$
 - Projection (π): Filters columns. Example: $\pi_{name, salary}(EMPLOYEE)$
 - Union ($\cup$): Combines rows from two compatible relations.
 - Set Difference ($-$): Rows in one relation but not in another.
 - Cartesian Product ($\times$): Combines every row of one relation with every row of another.
 - Rename (ρ): Renames a relation or its attributes.
 - **Additional Operations:**
 - Set Intersection ($\cap$)
 - Join ($\bowtie$): Combines related tuples from two relations.
 - Natural Join ($\bowtie_N$)
 - Theta Join ($\bowtie_\theta$)
 - Equijoin ($\bowtie_{\sim}$)
 - Division ($\div$)
 - **Relational Calculus:** Non-procedural query language.

- Tuple Relational Calculus (TRC)
- Domain Relational Calculus (DRC)
- **Structured Query Language (SQL):** Standard language for relational database management systems.
 - **DDL:** CREATE TABLE, ALTER TABLE, DROP TABLE.
 - **DML:** SELECT, INSERT INTO, UPDATE, DELETE FROM.
 - **DCL:** GRANT, REVOKE.
 - **TCL:** COMMIT, ROLLBACK, SAVEPOINT.
 - **Basic Queries:** SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...
 - **Joins:** INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN.
 - **Subqueries:** Nested queries.
 - **Views:** Virtual tables.

4. Database Design and Normalization

- **Constraints:** Rules that data in the database must follow.
 - **Domain Constraints:** Values must be from specified domain (data type, range).
 - **Referential Integrity:** If a foreign key in table A references a primary key in table B, then every value of the foreign key in A must either be null or exist as a value of the primary key in B.
 - **Entity Integrity:** Primary key attributes cannot be NULL.
- **Functional Dependencies ($X \rightarrow Y$):** If two tuples have the same values for attributes in X , then they must have the same values for attributes in Y .
 - **Trivial FD:** $Y \subseteq X$.
 - **Armstrong's Axioms:** Reflexivity, Augmentation, Transitivity. Used to infer FDs.
 - **Closure of a Set of FDs (F^+):** All FDs that can be inferred from F .
 - **Closure of Attributes (X^+):** Set of all attributes that are functionally determined by X .
- **Normalization:** Process of organizing the columns and tables of a relational database to minimize data redundancy and improve data integrity.
 - **1NF (First Normal Form):**
 - No repeating groups (multi-valued attributes).
 - Each column contains atomic (single, indivisible) values.
 - **2NF (Second Normal Form):**
 - Is in 1NF.
 - No non-prime attribute is partially dependent on any candidate key (i.e., depends on only part of a composite candidate key).
 - **3NF (Third Normal Form):**
 - Is in 2NF.
 - No non-prime attribute is transitively dependent on any candidate key (i.e., depends on another non-prime attribute).
 - **BCNF (Boyce-Codd Normal Form):**
 - Is in 3NF.
 - For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey.
 - Stricter than 3NF; handles cases where 3NF allows anomalies due to multiple candidate keys.

5. File Organization and Indexing

- **Operations on Files:** Creation, opening, closing, reading, writing, deleting.
- **Records:** Collection of related data items.
 - **Fixed-length Records:** All records have the same size. Easier to manage.
 - **Variable-length Records:** Records can have different sizes. More flexible, but complex to manage.
- **Sequential File Organization:** Records stored in physical sequence based on a key. Efficient for batch processing, slow for direct access.
- **Indexing Structures:** Data structures to speed up data retrieval operations on a database table.
 - **Primary Index:** Index on a field that is the primary key. Always dense.
 - **Secondary Index:** Index on a non-key field or a candidate key. Can be dense or sparse.
 - **Clustering Index:** Index where the order of records in the data file is the same as the order of the index entries. Only one per table.
 - **Dense Index:** Index entry for every search-key value in the data file.
 - **Sparse Index:** Index entry for only some of the search-key values.
- **B-Trees:** Self-balancing search trees.
 - Nodes can have more than two children.
 - All leaf nodes are at the same depth.
 - Efficient for disk-based storage systems due to reduced disk I/O.
- **B+ Trees:** Variation of B-trees, optimized for disk I/O.
 - All data pointers are stored only at the leaf nodes.
 - Leaf nodes are linked together, allowing for efficient range queries.
 - Internal nodes only store key values and pointers to child nodes.
- **Hashing:** Technique to directly compute the address of a data record on disk.
 - **Static Hashing:** Hash function maps keys to fixed set of addresses. Collisions handled by chaining or open addressing.

- **Dynamic Hashing:** Allows hash table to grow or shrink dynamically (e.g., Extendable Hashing, Linear Hashing).

6. Transaction Processing

- **Transaction:** A single logical unit of work, consisting of one or more database operations.
- **ACID Properties:** Fundamental properties of database transactions.
 - **Atomicity:** All or nothing. Either all operations in a transaction are completed, or none are.
 - **Consistency:** A transaction must bring the database from one valid state to another.
 - **Isolation:** Concurrent execution of transactions leaves the database in the same state as if the transactions were executed sequentially.
 - **Durability:** Once a transaction is committed, its changes are permanent and survive system failures.
- **Concurrency Control:** Mechanisms to ensure that concurrent transactions execute correctly without interfering with each other.
 - **Locking Protocols:**
 - Shared (S) locks: Allow multiple readers.
 - Exclusive (X) locks: Allow only one writer.
 - Two-Phase Locking (2PL): Growing phase (acquire locks), Shrinking phase (release locks). Guarantees serializability.
 - **Timestamp-Based Protocols:** Assigns a unique timestamp to each transaction.
 - **Validation-Based (Optimistic) Protocols:** Transactions execute without locks, then validated before commit.
- **Recovery:** Mechanisms to restore the database to a consistent state after a system failure.
 - **Log-based Recovery:** Uses a log file to record all database modifications.
 - **Checkpoints:** Periodically records current state to speed up recovery.

7. Distributed Databases

- **Introduction to Distributed Databases:** A database that is stored on several computers, located in different geographical locations, and connected by a computer network.
- **Comparison with Traditional (Centralized) Databases:**
 - **Advantages:** Data sharing, higher reliability/availability, improved performance, easier expansion.
 - **Disadvantages:** Increased complexity, higher cost, complex security issues, integrity control.
- **DDBMS Components:**
 - **Global Schema:** Integrated logical view of the entire database.
 - **Local Schemas:** View of data stored at each site.
 - **Distributed Data Dictionary/Directory:** Stores metadata about the distributed database.
 - **Distributed Query Processor:** Optimizes and executes queries across multiple sites.
 - **Distributed Transaction Manager:** Ensures ACID properties across sites (e.g., Two-Phase Commit).
- **Transparency in DDBMS:**
 - **Distribution Transparency:** Users don't need to know where data is located.
 - **Replication Transparency:** Users don't need to know if data is replicated.
 - **Fragmentation Transparency:** Users don't need to know how data is fragmented.
- **Data Fragmentation:** Dividing a relation into smaller fragments.
 - **Horizontal Fragmentation:** Dividing by rows (subsets of tuples).
 - **Vertical Fragmentation:** Dividing by columns (subsets of attributes).
 - **Mixed Fragmentation:** Combination of horizontal and vertical.
- **Data Replication:** Storing copies of data fragments at multiple sites for availability and performance.